

Clase 3

Cómo crear modelos de datos de aprendizaje

Guía de trabajo del aprendiz · De los datos a un modelo que predice

Duración: 4 horas (8:00 am – 12:00 m)

Pablo Andrés Carvajal — Desarrollador Full Stack

pablocarvajal.dev

Nombre del aprendiz: _____

¿Qué sabré hacer al terminar esta guía?

- Explicar qué es un buen **dataset** y las etapas del ciclo de vida de los datos.
- Identificar y corregir problemas comunes de **limpieza** de datos.
- Distinguir **features** y **target**, y preparar los datos (one-hot, normalización).
- Dividir los datos en **entrenamiento** y **prueba** y explicar por qué.
- Entrenar, **evaluar** (exactitud y matriz de confusión) y detectar **overfitting**.
- Predecir sobre datos nuevos y **guardar** el modelo con joblib.

CÓMO USAR ESTA GUÍA

Sigue el contenido teórico y luego reproduce el proyecto paso a paso en tu computador. El código completo y comentado está en [clase-3/codigo/entrenar_modelo.py](#). Al final encontrarás actividades de refuerzo y una lista de verificación.

DEMOS INTERACTIVOS DE ESTA CLASE

Abre estas páginas en el navegador para **vivir** el ciclo de los datos:

- **Entrenar y evaluar un modelo sin código** (con tu cámara): [Teachable Machine](#)
- **Sobreajuste y división train/test (visual)**: [R2D3 parte 1](#) y [parte 2](#)
- **Datasets reales para practicar**: [Kaggle Datasets](#)

Todas juntas en el hub del curso: [recursos/demos-interactivos.html](#)

1. El dato es el rey

"Garbage in, garbage out" (basura entra, basura sale): un modelo nunca es mejor que los datos con los que aprende. En un proyecto real de IA, alrededor del **80% del trabajo** es recolectar, limpiar y preparar los datos; entrenar el modelo es solo una parte pequeña.

1.1 Ciclo de vida de un proyecto de datos

1. RECOLECTAR → 2. EXPLORAR → 3. LIMPIAR → 4. PREPARAR (features)
→ 5. ENTRENAR → 6. EVALUAR → 7. DESPLEGAR / USAR → (volver a 1)

Es un **ciclo**, no una línea recta: cuando el modelo se usa en el mundo real aparecen datos nuevos y aprendizajes que alimentan una versión mejorada.

1.2 ¿Qué es un buen dataset?

- **Suficiente cantidad:** más ejemplos permiten que el modelo generalice mejor.
- **Representativo:** debe parecerse a los casos reales donde se va a usar.
- **Balanceado:** si el 99% de los ejemplos son de una clase, el modelo aprenderá a decir siempre esa clase.
- **Sin sesgos dañinos:** si los datos contienen prejuicios, el modelo los aprende y los amplifica.
- **Limpio:** sin errores, duplicados ni valores faltantes descontrolados.

1.3 Problemas comunes al limpiar datos

Problema	Ejemplo	Qué hacer
Valores faltantes	Edad = (vacío)	Rellenar (media/mediana) o eliminar la fila
Duplicados	La misma fila 2 veces	Eliminar los repetidos
Formatos mezclados	"Sí", "si", "SI", "1"	Estandarizar a un único valor
Valores atípicos (outliers)	Edad = 999	Revisar y corregir o descartar
Texto en vez de número	"\$1.200.000"	Convertir a número (1200000)

2. Preparación de datos: features y división

2.1 Features y target

- **Features (X):** las columnas de entrada que describen cada ejemplo.
- **Target (y):** lo que queremos predecir.

El modelo aprende la relación entre **X** y **y** a partir de muchos ejemplos. En el proyecto de hoy, las features son las medidas de la flor y el target es la especie.

2.2 Ingeniería de features (feature engineering)

Consiste en crear o transformar columnas para ayudar al modelo:

- **Variables categóricas → números (one-hot encoding):** "rojo/verde/azul" no se puede sumar. Se crea una columna 0/1 por categoría, de modo que el modelo pueda trabajarlas.
- **Escalado / normalización:** poner todas las features en rangos parecidos (por ejemplo, entre 0 y 1) para que ninguna domine solo por tener números grandes.
- **Fechas → partes útiles:** de "2026-09-04" extraer día de la semana, mes, etc.

NOTA

El árbol de decisión que usaremos hoy no necesita normalización, pero muchos modelos (regresión, redes neuronales, k-NN) sí. Es un buen hábito conocerla.

2.3 La división más importante: entrenamiento vs. prueba

Nunca evaluamos el modelo con los mismos datos con los que aprendió (¡sería como darle el examen con las respuestas!). Por eso dividimos el dataset:

DATASET COMPLETO (100%)

ENTRENAMIENTO (70-80%) el modelo aprende aquí	PRUEBA (20-30%) examen final
--	---------------------------------

- **Train:** el modelo ajusta sus parámetros internos.
- **Test:** medimos qué tan bien generaliza a datos que **nunca vio**.

OVERFITTING

Si al modelo le va muy bien en train pero mal en test, significa que **memorizó** en lugar de aprender. Soluciones: más datos, un modelo más simple (menor `max_depth`) o regularización.

3. Proyecto guiado: clasificar flores Iris

Usaremos el dataset clásico **Iris**: medidas de flores para predecir su especie. Es pequeño, limpio y perfecto para aprender el flujo completo. Instala las librerías:

```
pip install scikit-learn pandas joblib
```

El código completo está en [clase-3/codigo/entrenar_modelo.py](#). A continuación lo explicamos por partes.

Paso 1 — Cargar y explorar los datos

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names) # features (entradas)
y = iris.target # target (0,1,2 = especies)

print(X.head()) # primeras filas
print("Especies:", list(iris.target_names))
print("Total de ejemplos:", len(X))
```

Antes de entrenar conviene **mirar los datos**: cuántas filas hay, qué columnas (features) los describen y cuál es el target. Iris tiene 150 ejemplos, 4 features (largo y ancho de sépalo y pétalo) y 3 especies: *setosa*, *versicolor* y *virginica*.

Paso 2 — Dividir en entrenamiento y prueba

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42 # 20% para prueba
)
print("Entrenamiento:", len(X_train), " | Prueba:", len(X_test))
```

`test_size=0.2` reserva el 20% de los datos para el examen final. `random_state=42` fija el "azar" de la división para que los resultados sean **reproducibles** (siempre la misma partición).

Paso 3 — Elegir y entrenar el modelo

Usamos un **árbol de decisión**: aprende reglas del tipo "si el pétalo mide más de X → especie Y". Es fácil de entender e interpretar.

```
from sklearn.tree import DecisionTreeClassifier

modelo = DecisionTreeClassifier(max_depth=3, random_state=42)
modelo.fit(X_train, y_train)  # <- aquí el modelo APRENDE
```

.FIT() = ENTRENAMIENTO

La línea `modelo.fit(...)` es el momento en que el modelo mira los ejemplos y ajusta sus reglas internas. `max_depth=3` limita la profundidad del árbol para que no se vuelva demasiado complejo (menos overfitting).

4. Evaluar el modelo

Paso 4 — Medir qué tan bien predice

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

predicciones = modelo.predict(X_test)  # predice sobre datos NUEVOS

exactitud = accuracy_score(y_test, predicciones)
print(f"Exactitud: {exactitud:.2%}")      # ej. 96.67%

print("\nMatriz de confusión:")
print(confusion_matrix(y_test, predicciones))

print("\nReporte detallado:")
print(classification_report(y_test, predicciones, target_names=iris.target_names))
```

4.1 Cómo leer las métricas

- **Exactitud (accuracy)**: porcentaje de predicciones correctas. Es simple, pero **engaña** cuando los datos están desbalanceados.
- **Precisión y recall**: útiles cuando importa más un tipo de error que otro (por ejemplo, en un diagnóstico médico un falso negativo puede ser grave).

4.2 Cómo leer una matriz de confusión

La matriz de confusión es una tabla que cruza lo **real** (filas) contra lo que el modelo **predijo** (columnas). Cada celda cuenta cuántos ejemplos cayeron ahí:

```

                Predijo
                setosa versí virgin
Real setosa [ 10   0   0 ] ← 10 aciertos
     versí  [  0   9   1 ] ← confundió 1 versicolor con virginica
     virgin [  0   0  10 ]
```

- La **diagonal** (de arriba-izquierda a abajo-derecha) son los **aciertos**: lo real coincide con lo predicho.
- Todo lo que está **fuera de la diagonal** son **errores**, y muestra exactamente **en qué se confunde** el modelo.
- En el ejemplo, el modelo acertó 29 de 30 flores; solo confundió 1 versicolor con virginica.

TRUCO

Lee cada fila así: "de las N flores que *realmente* eran de esta especie, ¿cuántas predijo bien y con cuáles las confundió?".

Paso 4b — Detectar overfitting

```
print("Exactitud en TRAIN:", modelo.score(X_train, y_train))
print("Exactitud en TEST: ", modelo.score(X_test, y_test))
```

Si la exactitud en TRAIN es cercana al 100% pero en TEST es mucho menor, el modelo está **sobreajustado** (overfitting): memorizó los ejemplos de entrenamiento en vez de aprender patrones generales. Prueba a bajar `max_depth`, conseguir más datos o usar un modelo con regularización.

5. Usar y guardar el modelo

Paso 5 — Predecir sobre una flor nueva

```
import pandas as pd

flor_nueva = pd.DataFrame([[5.1, 3.5, 1.4, 0.2]], columns=iris.feature_names)
pred = modelo.predict(flor_nueva)
print("Especie predicha:", iris.target_names[pred[0]])
```

Le entregamos medidas que el modelo **nunca vio** y responde con la especie más probable. Así es como el modelo "trabaja" una vez entrenado.

Paso 6 — Guardar el modelo entrenado

```
import joblib

joblib.dump(modelo, "modelo_iris.pkl")          # guardar
# ... en otro programa:
modelo_cargado = joblib.load("modelo_iris.pkl")  # cargar y usar
```

LLEVAR A PRODUCCIÓN

Guardar el modelo evita re-entrenarlo cada vez. Se entrena una sola vez, se guarda en un archivo `.pkl`, y una aplicación lo carga para responder al instante. Este es el puente entre "experimentar" y "usar en el mundo real".

Actividades de refuerzo

Realiza estas actividades para afianzar lo aprendido. Puedes escribir tus respuestas en los espacios en blanco o anexar capturas de tu código y resultados.

1 Ciclo de vida con mis palabras Fácil

Enumera las 7 etapas del ciclo de vida de los datos y explica en una frase qué se hace en cada una.

2 Detective de datos sucios Fácil

Para cada problema de limpieza, escribe un ejemplo propio (distinto a los de la guía) y cómo lo corregirías: valor faltante, duplicado, formato mezclado, outlier y texto en vez de número.

3 Ejecuta el proyecto Iris Media

Ejecuta `python entrenar_modelo.py` (dentro de `clase-3/codigo/`). Anota la exactitud en test que obtuviste y copia la matriz de confusión. ¿En qué especie se confunde más el modelo?

4 Experimenta con max_depth Media

Cambia `max_depth` a 1 y luego a 10. Completa la tabla con la exactitud en train y en test para cada valor. ¿Con cuál aparece overfitting y cómo lo notas?

max_depth	Exactitud TRAIN	Exactitud TEST
1		
3		
10		

5 Prueba un bosque de árboles Reto

Reemplaza el `DecisionTreeClassifier` por `RandomForestClassifier` (`from sklearn.ensemble import RandomForestClassifier`). Entrena y evalúa igual. ¿Mejora la exactitud en test respecto al árbol simple? Anota los dos resultados y compáralos.

6 Tu propio CSV Reto

Con ayuda de **OpenCode**, carga un CSV propio (ventas, notas, clientes, etc.) con `pd.read_csv("archivo.csv")` , elige una columna como target, adapta el flujo (limpieza, train/test, entrenar, evaluar) y predice sobre una fila nueva. Describe tu dataset y el resultado.

Lista de verificación

- ☐ Sé nombrar las etapas del ciclo de vida de los datos.

- ☐ Puedo identificar y corregir problemas comunes de limpieza.
- ☐ Distingo features (X) del target (y).
- ☐ Entiendo por qué se separan datos de train y test.
- ☐ Puedo entrenar y evaluar un modelo con scikit-learn.
- ☐ Sé leer una matriz de confusión y detectar overfitting.
- ☐ Sé guardar y reutilizar un modelo con joblib.

SOLUCIÓN DE PROBLEMAS

ModuleNotFoundError: ejecuta `pip install scikit-learn pandas joblib` . · **pip no se reconoce:** usa `python -m pip install ...` · **Resultados distintos cada vez:** revisa que `random_state=42` esté puesto. · **Exactitud muy baja:** revisa la limpieza de tus datos y la elección del target.